

## PANDUAN MEMBUAT CRUD BUKUS DI LARAVEL

### PENGENALAN

CRUD adalah singkatan dari **\*\*Create, Read, Update, Delete\*\*** - empat operasi dasar dalam manajemen data. Panduan ini akan menjelaskan langkah-langkah lengkap untuk membuat fitur CRUD untuk tabel buku di aplikasi perpustakaan Laravel.

---

### TAHAP 1: PERSIAPAN DATABASE

#### 1.1 MEMBUAT MIGRATION (MIGRASI DATABASE)

Migration digunakan untuk membuat struktur tabel di database secara terprogram.

**\*\*File\*\***: 'database/migrations/2026\_02\_11\_081154\_create\_bukus\_table.php'

<?php

```
use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
```

```
public function up(): void
```

```
{
```

```
Schema::create('bukus', function (Blueprint $table) {
```

```
$table->id(); // ID otomatis
```

```
$table->string('judul'); // Judul buku
```

```
$table->string('penulis'); // Nama penulis
```

```
$table->string('tahun_terbit'); // Tahun terbit
```

```
$table->foreignId('kategori_id') // Foreign key ke tabel kategori
```

```
->constrained('kategori');
```

```
$table->integer('stok'); // Jumlah stok buku
```

```
$table->timestamps(); // Kolom created_at dan updated_at
```

```
});
```

```
}
```

```
public function down(): void
```

```
{
```

```
Schema::dropIfExists('bukus');
```

```
}
```

```
};
```

**\*\*Penjelasan\*\***:

â € 'id()': Membuat kolom ID sebagai primary key dengan auto increment

â € 'string()': Membuat kolom teks dengan panjang 255 karakter

â € 'integer()': Membuat kolom untuk angka bulat

â € 'foreignId()': Membuat kolom yang mereferensikan tabel lain (foreign key)

â € 'timestamps()': Membuat kolom 'created\_at' dan 'updated\_at' otomatis

â € 'constrained()': Menentukan nama tabel yang direferensikan

**\*\*Cara Menjalankan\*\***:

```
php artisan migrate
```

---

### TAHAP 2: MEMBUAT MODEL

#### 2.1 MODEL BUKU

Model adalah representasi data yang menghubungkan aplikasi dengan database.

**\*\*File\*\***: 'app/Models/Buku.php'

<?php

```
namespace App\Models;
```

```

use Illuminate\Database\Eloquent\Model;
class Buku extends Model
{
    // Kolom yang dapat diisi mass assignment
    protected $fillable = ['judul', 'penulis', 'tahun_terbit', 'kategori_id', 'stok'];
    // Relasi Many-to-One dengan Kategori
    public function kategori()
    {
        return $this->belongsTo(Kategori::class);
    }
}

**Penjelasan**:
â € $ 'fillable': Mendaftarkan kolom mana saja yang boleh diisi melalui 'create()' atau
'update()' untuk keamanan
â € $ 'belongsTo()': Menunjukkan bahwa satu buku milik satu kategori
---
```

### TAHAP 3: MEMBUAT CONTROLLER

#### 3.1 CONTROLLER UNTUK CRUD BUKUS

Controller menangani logika bisnis dan mengatur alur data.

**\*\*File\*\***: 'app/Http/Controllers/BukuController.php'

```

<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Buku;
use App\Models\Kategori;
class BukuController extends Controller
{
    /**
     â € READ - Menampilkan daftar semua buku
    */
    public function index()
    {
        $bukus = Buku::with('kategori')->get(); // Ambil semua buku dengan relasi kategori
        return view('bukus.index', compact('bukus'));
    }
    /**
     â € CREATE (Bagian 1) - Menampilkan form tambah buku
    */
    public function create()
    {
        $kategoris = Kategori::all(); // Ambil semua kategori untuk dropdown
        return view('bukus.create', compact('kategoris'));
    }
    /**
     â € CREATE (Bagian 2) - Menyimpan data buku ke database
    */
    public function store(Request $request)
    {
        // Validasi input dari form
        $request->validate([
```

```

'judul' => 'required', // Harus diisi
'penulis' => 'required',
'tahun_terbit' => 'required|integer', // Harus berupa angka
'kategori_id' => 'required|exists:kategoris,id', // Harus ada di tabel kategoris
'stok' => 'required|integer',
]);
// Simpan data ke database
Buku::create($request->all());

// Redirect ke halaman daftar dengan pesan sukses
return redirect()->route('bukus.index')->with('success', 'Buku berhasil ditambahkan');
}
/**
 * READ (Detail) - Menampilkan detail satu buku
 */
public function show(Buku $buku)
{
    return view('bukus.show', compact('buku'));
}
/**
 * UPDATE (Bagian 1) - Menampilkan form edit buku
 */
public function edit(Buku $buku)
{
    $kategoris = Kategori::all();
    return view('bukus.edit', compact('buku', 'kategoris'));
}
/**
 * UPDATE (Bagian 2) - Menyimpan perubahan data buku
 */
public function update(Request $request, Buku $buku)
{
    // Validasi input
    $request->validate([
        'judul' => 'required',
        'penulis' => 'required',
        'tahun_terbit' => 'required|integer',
        'kategori_id' => 'required|exists:kategoris,id',
        'stok' => 'required|integer',
    ]);
    // Update data di database
    $buku->update($request->all());

    return redirect()->route('bukus.index')->with('success', 'Buku berhasil diupdate');
}
/**
 * DELETE - Menghapus data buku dari database
 */
public function destroy(Buku $buku)
{

```

```

$buku->delete();
return redirect()->route('bukus.index')->with('success', 'Buku berhasil dihapus');
}
}
**Penjelasan Method**:
â € 'index()': Menampilkan list semua buku (READ)
â € 'create()': Menampilkan form untuk membuat buku baru (CREATE - tampilan)
â € 'store()': Menyimpan data buku baru ke database (CREATE - simpan)
â € 'show()': Menampilkan detail satu buku (READ detail)
â € 'edit()': Menampilkan form untuk edit buku (UPDATE - tampilan)
â € 'update()': Menyimpan perubahan buku ke database (UPDATE - simpan)
â € 'destroy()': Menghapus buku dari database (DELETE)

```

---

## TAHAP 4: MEMBUAT ROUTES (ROUTING)

### 4.1 MENDAFTARKAN ROUTE RESOURCE

Routes menghubungkan URL dengan controller method.

**\*\*File\*\***: 'routes/web.php'

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
use App\Http\Controllers\BukuController;
```

```
// Route resource otomatis membuat 7 route untuk CRUD
```

```
Route::resource('bukus', BukuController::class);
```

**\*\*Route yang Dibuat Otomatis\*\***:

Metode HTTP URL Method Deskripsi

-----

GET /bukus index Tampilkan daftar buku

GET /bukus/create create Tampilkan form tambah

POST /bukus store Simpan buku baru

GET /bukus/{id} show Tampilkan detail buku

GET /bukus/{id}/edit edit Tampilkan form edit

PUT/PATCH /bukus/{id} update Simpan perubahan buku

DELETE /bukus/{id} destroy Hapus buku

---

## TAHAP 5: MEMBUAT VIEWS (TAMPILAN)

### 5.1 INDEX VIEW - DAFTAR BUKU

**\*\*File\*\***: 'resources/views/bukus/index.blade.php'

```
@extends('layouts.app')
```

```
@section('content')
```

```
<h3>Data Buku</h3>
```

```
<!-- Tombol Tambah Buku -->
```

```
<a href="{{ route('bukus.create') }}" class="btn btn-primary mb-3">Tambah Buku</a>
```

```
<!-- Tabel Daftar Buku -->
```

```
<table class="table table-bordered">
```

```
<thead>
```

```
<tr>
```

```
<th>Judul</th>
```

```
<th>Penulis</th>
```

```
<th>Tahun Terbit</th>
```

```
<th>Kategori</th>
```

```
<th>Stok</th>
```

```

<th>Aksi</th>
</tr>
</thead>
<tbody>
@foreach($bukus as $buku)
<tr>
<td>{{ $buku->judul }}</td>
<td>{{ $buku->penulis }}</td>
<td>{{ $buku->tahun_terbit }}</td>
<!-- Tampilkan nama kategori dari relasi -->
<td>{{ $buku->kategori->nama_kategori ?? 'N/A' }}</td>
<td>{{ $buku->stok }}</td>
<td>
<!-- Tombol Lihat Detail -->
<a href="{{ route('bukus.show', $buku) }}" class="btn btn-info btn-sm">Lihat</a>

<!-- Tombol Edit -->
<a href="{{ route('bukus.edit', $buku) }}" class="btn btn-warning btn-sm">Edit</a>

<!-- Tombol Hapus dengan Konfirmasi -->
<form action="{{ route('bukus.destroy', $buku) }}" method="POST"
style="display:inline;">
@csrf
@method('DELETE')
<button type="submit" class="btn btn-danger btn-sm" onclick="return confirm('Apakah
Anda yakin?')">Hapus</button>
</form>
</td>
</tr>
@endforeach
</tbody>
</table>
@endsection
**Penjelasan**:
â € '@extends()': Mewarisi template layout
â € '@section()': Mendefinisikan bagian konten
â € '{{ route() }}': Membuat URL dari nama route
â € '{{ $buku->judul }}': Menampilkan data dari model
â € '@foreach()': Perulangan untuk menampilkan list
â € '@csrf': Token keamanan Laravel
â € '@method()': Untuk method HTTP selain GET/POST
5.2 CREATE VIEW - FORM TAMBAH BUKU
**File**: 'resources/views/bukus/create.blade.php'
@extends('layouts.app')
@section('content')
<h3>Tambah Buku</h3>
<form action="{{ route('bukus.store') }}" method="POST">
@csrf

<div class="form-group">

```

```

<label>Judul</label>
<input type="text" name="judul" class="form-control" required>
</div>

<div class="form-group">
<label>Penulis</label>
<input type="text" name="penulis" class="form-control" required>
</div>

<div class="form-group">
<label>Tahun Terbit</label>
<input type="number" name="tahun_terbit" class="form-control" required>
</div>

<div class="form-group">
<label>Kategori</label>
<select name="kategori_id" class="form-control" required>
@foreach($kategoris as $kategori)
<option value="{{ $kategori->id }}">{{ $kategori->nama_kategori }}</option>
@endforeach
</select>
</div>

<div class="form-group">
<label>Stok</label>
<input type="number" name="stok" class="form-control" required>
</div>

<button type="submit" class="btn btn-primary">Simpan</button>
<a href="{{ route('bukus.index') }}" class="btn btn-secondary">Kembali</a>
</form>
@endsection
**Penjelasan**:
â ¤ Form action mengarah ke 'bukus.store' (method POST)
â ¤ '@csrf': Token keamanan untuk mencegah CSRF attack
â ¤ Input fields sesuai dengan kolom tabel di database
â ¤ Dropdown kategori di-populate dari data kategori
5.3 EDIT VIEW - FORM EDIT BUKU
**File**: 'resources/views/bukus/edit.blade.php'
@extends('layouts.app')
@section('content')
<h3>Edit Buku</h3>
<form action="{{ route('bukus.update', $buku) }}" method="POST">
@csrf
@method('PUT') <!-- Menentukan method PUT untuk update -->

<div class="form-group">
<label>Judul</label>
<input type="text" name="judul" value="{{ $buku->judul }}" class="form-control"
required>

```

```
</div>
```

```
<div class="form-group">
<label>Penulis</label>
<input type="text" name="penulis" value="{{ $buku->penulis }}" class="form-control"
required>
</div>
```

```
<div class="form-group">
<label>Tahun Terbit</label>
<input type="number" name="tahun_terbit" value="{{ $buku->tahun_terbit }}"
class="form-control" required>
</div>
```

```
<div class="form-group">
<label>Kategori</label>
<select name="kategori_id" class="form-control" required>
@foreach($kategoris as $kategori)
<option value="{{ $kategori->id }}" {{ $buku->kategori_id == $kategori->id ?
'selected' : '' }}>
{{ $kategori->nama_kategori }}
</option>
@endforeach
</select>
</div>
```

```
<div class="form-group">
<label>Stok</label>
<input type="number" name="stok" value="{{ $buku->stok }}" class="form-control"
required>
</div>
```

```
<button type="submit" class="btn btn-primary">Update</button>
<a href="{{ route('bukus.index') }}" class="btn btn-secondary">Kembali</a>
</form>
```

```
@endsection
```

```
**Penjelasan**:
```

â ¢ Form action mengarah ke 'bukus.update' (method PUT)

â ¢ '@method('PUT)': Karena form HTML hanya support GET/POST

â ¢ 'value="{{ \$buku->judul }}"': Pre-fill form dengan data yang ada

â ¢ '{{ \$buku->kategori\_id == \$kategori->id ? 'selected' : '' }}': Pilih kategori yang sesuai

#### 5.4 SHOW VIEW - DETAIL BUKU

```
**File**: 'resources/views/bukus/show.blade.php'
```

```
@extends('layouts.app')
```

```
@section('content')
```

```
<h3>Detail Buku</h3>
```

```
<div class="card">
```

```
<div class="card-body">
```

```
<h5 class="card-title">{{ $buku->judul }}</h5>
```

```

<p><strong>Penulis:</strong> {{ $buku->penulis }}</p>
<p><strong>Tahun Terbit:</strong> {{ $buku->tahun_terbit }}</p>
<p><strong>Kategori:</strong> {{ $buku->kategori->nama_kategori ?? 'N/A' }}</p>
<p><strong>Stok:</strong> {{ $buku->stok }}</p>
</div>
</div>
<a href="{{ route('bukus.index') }}" class="btn btn-secondary">Kembali</a>
@endsection

```

---

## TAHAP 6: ALUR KERJA CRUD

### 6.1 CREATE (MEMBUAT DATA BARU)

**\*\*Alur\*\*:**

1. User klik tombol "Tambah Buku" di halaman index
2. Browser navigasi ke '/bukus/create' (GET request)
3. Controller method 'create()' dijalankan
4. Form HTML ditampilkan ke user
5. User isi form dan klik "Simpan"
6. Form di-submit ke '/bukus' (POST request)
7. Controller method 'store()' dijalankan
8. Data divalidasi
9. Data disimpan ke database dengan 'Buku::create()'
10. User redirect ke halaman index dengan pesan sukses

### 6.2 READ (MEMBACA DATA)

**\*\*Daftar Buku (Index)\*\*:**

1. User akses '/bukus'
2. Controller method 'index()' dijalankan
3. Data diambil dari database: 'Buku::with('kategori')->get()'
4. Data ditampilkan di tabel

**\*\*Detail Buku (Show)\*\*:**

1. User klik tombol "Lihat" di tabel
2. Browser navigasi ke '/bukus/{id}' (GET request)
3. Controller method 'show()' dijalankan dengan model binding
4. Detail buku ditampilkan

### 6.3 UPDATE (MENGUBAH DATA)

**\*\*Alur\*\*:**

1. User klik tombol "Edit" di tabel
2. Browser navigasi ke '/bukus/{id}/edit' (GET request)
3. Controller method 'edit()' dijalankan
4. Form ditampilkan dengan data yang sudah ada (pre-fill)
5. User ubah data dan klik "Update"
6. Form di-submit ke '/bukus/{id}' (PUT request)
7. Controller method 'update()' dijalankan
8. Data divalidasi
9. Data diperbarui di database dengan '\$buku->update()'
10. User redirect dengan pesan sukses

### 6.4 DELETE (MENGHAPUS DATA)

**\*\*Alur\*\*:**

1. User klik tombol "Hapus" di tabel
2. Browser menampilkan konfirmasi dialog
3. Jika user konfirmasi, form di-submit dengan method DELETE



4. Browser mengirim DELETE request ke '/bukus/{id}'
5. Controller method 'destroy()' dijalankan
6. Data dihapus dari database dengan '\$buku->delete()'
7. User redirect dengan pesan sukses

---

## TAHAP 7: FITUR KEAMANAN

### 7.1 CSRF PROTECTION

Semua form dalam Laravel dilindungi dari CSRF attack dengan token:

```
<form action="{{ route('bukus.store') }}" method="POST">
```

```
@csrf <!-- Token security -->
```

```
...
```

```
</form>
```

### 7.2 VALIDASI DATA

Data divalidasi sebelum disimpan ke database:

```
$request->validate([  
    'judul' => 'required',  
    'tahun_terbit' => 'required|integer',  
    'kategori_id' => 'required|exists:kategori,id',  
]);
```

### 7.3 MASS ASSIGNMENT PROTECTION

Model hanya mengizinkan kolom tertentu untuk diisi:

```
class Buku extends Model  
{  
    protected $fillable = ['judul', 'penulis', 'tahun_terbit', 'kategori_id', 'stok'];  
}
```

---

## TAHAP 8: TESTING CRUD

### TEST CREATE:

1. Akses '/bukus/create'
2. Isi form dengan data buku baru
3. Klik "Simpan"
4. Verifikasi data muncul di halaman index dan database

### TEST READ:

1. Klik tombol "Lihat" pada salah satu buku
2. Verifikasi detail buku ditampilkan dengan benar

### TEST UPDATE:

1. Klik tombol "Edit" pada salah satu buku
2. Ubah salah satu field
3. Klik "Update"
4. Verifikasi perubahan tersimpan di database

### TEST DELETE:

1. Klik tombol "Hapus" pada salah satu buku
2. Konfirm di dialog
3. Verifikasi buku terhapus dari list dan database

---

## RINGKASAN FILE-FILE YANG DIBUAT

Komponen File

-----  
Migration 'database/migrations/2026\_02\_11\_081154\_...'

Model 'app/Models/Buku.php'

Controller 'app/Http/Controllers/BukuController.php'  
Routes 'routes/web.php'  
View Index 'resources/views/bukus/index.blade.php'  
View Create 'resources/views/bukus/create.blade.php'  
View Edit 'resources/views/bukus/edit.blade.php'  
View Show 'resources/views/bukus/show.blade.php'  
---

#### KESIMPULAN

CRUD adalah fondasi aplikasi web modern. Dengan memahami alur CRUD:

â € \*\*CREATE\*\*: Tambahkan data baru ke database

â € \*\*READ\*\*: Tampilkan data dari database

â € \*\*UPDATE\*\*: Ubah data yang sudah ada

â € \*\*DELETE\*\*: Hapus data yang tidak diperlukan

Anda dapat membangun aplikasi yang efisien dan terstruktur. Laravel menyediakan tools seperti Eloquent ORM, Blade templating, dan Resource routing yang membuat pembuatan CRUD menjadi cepat dan mudah.